

INTERNSHIP PROJECT REPORT

IoT and Smart Technologies

ServeIQ

A Multi-Portal Cloud-Based Hotel & Restaurant
Management SaaS Platform

Pravidhi Digital Innovations Nepal Pvt Ltd

August 2026

CONFIDENTIAL — FOR INTERNAL EVALUATION ONLY

Table of Contents

1. Project Title and Introduction

- 1.1 Project Title
- 1.2 Introduction and Background
- 1.3 Connection to IoT and Smart Technologies
- 1.4 Organization

2. Problem Statement

- 2.1 Existing Problem
- 2.2 How ServeIQ Addresses the Problem

3. System Workflow / Block Diagram

- 3.1 High-Level System Architecture
- 3.2 Booking Flow Workflow
- 3.3 Check-in / Check-out Flow
- 3.4 Restaurant POS / KDS Flow

4. Description of Components / Modules

- 4.1 Frontend
- 4.2 Backend
- 4.3 Database
- 4.4 Payment Gateway
- 4.5 Operations Portal
- 4.6 Host Portal
- 4.7 SuperAdmin Portal
- 4.8 Guest Portal
- 4.9 Push Notifications

4.10 API Client

5. Student's Role and Contribution

5.1 Overview

5.2 What I Did

5.3 Testing and Verification

6. Summary and Conclusion

6.1 Project Summary

6.2 Effectiveness

6.3 Smart Technologies Connection

6.4 Skills Gained

6.5 Conclusion

1. Project Title and Introduction

1.1 Project Title

ServeIQ — A Multi-Portal Cloud-Based Hotel & Restaurant Management SaaS Platform

1.2 Introduction and Background

I have been doing my internship in Pravidhi Digital Innovations Pvt. Ltd. I am currently working as a Mobile Application Developer on the SaaS project of ServeIQ. Basically I build the mobile app side of things.

"If I had to explain it to my friend I would just say it is a more advanced form of Booking.com. But it also includes restaurant POS, kitchen display system, and you can list any property using a simple interface. Also it works for hosts as well, not just guests."

So ServeIQ is basically a hotel and restaurant management system. It is a cloud based platform where different types of users like hotel owners, their staff, and guests can all use the same app but they see different things based on who they are. The whole idea is to replace the old way of managing hotels which is mostly paper based or uses separate tools that do not talk to each other.

The app is built with **Expo SDK 57 and React Native** using TypeScript. This means one codebase runs on iOS, Android, and web. The backend is **FastAPI Python** hosted on **Render** and the database is **PostgreSQL**. The whole system talks through REST APIs.

There are four portals in the app. The **Guest portal** is for people who want to book rooms. The **Host portal** is for property owners who manage their listings. The **Operations portal** is for staff like front desk, housekeeping, restaurant, kitchen. And the **SuperAdmin portal** is for the people who run the whole platform. Each portal has its own color theme and its own login.

1.3 Connection to IoT and Smart Technologies

How This Connects to IoT and Smart Technologies:

Even though ServeIQ is a software app and not a physical IoT device it uses some ideas that are similar to smart systems. The GPS based nearby search uses the phone's location to show relevant results which is a location aware service like how smart systems adapt based on where you are. Push notifications happen automatically when something changes like a new booking comes in or a room gets dirty or kitchen finishes an order. This is event driven like how sensors trigger actions in IoT. The room status follows a fixed flow which is Available then Occupied then Dirty then Cleaned then Inspected then back to Available. That is a state machine which is the same logic used in smart buildings and automation systems. So even though we are not dealing with sensors and actuators the way the system works is built on similar principles.

1.4 Organization

I am doing this internship at **Pravidhi Digital Innovations Nepal Pvt Ltd.** The company builds software for hotels and restaurants in Nepal. They focus on making affordable tools for small and medium businesses that cannot afford expensive enterprise systems.

2. Problem Statement

2.1 Existing Problem

Hotels and restaurants in Nepal still use a lot of manual processes. Here are the main problems I noticed.

- 1. Everything is separate.** The front desk uses one system the kitchen uses paper tickets and housekeeping finds out about dirty rooms by walking around. None of these systems talk to each other. This causes a lot of confusion and missed information.
- 2. Double bookings happen a lot.** When a guest books by phone another one walks in and someone else books online there is no single place checking if the room is actually available. So you end up with two people booked for the same room.
- 3. Staff communication is slow.** When a guest checks out the housekeeping team does not know until someone tells them. In the restaurant waiters write orders on paper and hand them to the kitchen. Orders get lost or written wrong.
- 4. No way to set smart pricing.** Hotel owners charge the same rate all year round. They cannot easily change prices for peak season or festivals or weekends. They also cannot offer discounts or deals in a structured way.
- 5. Bad guest experience.** Guests have to call the hotel to book. There is no app where they can search see photos compare prices and book instantly. Everything happens over phone or email.
- 6. No reports or analytics.** Owners do not know their occupancy rate or how much revenue they are making per room or which months are busiest. They just guess based on memory.
- 7. Cannot scale.** Most existing tools work for one property only. If someone owns two or three hotels they need separate systems for each one.

2.2 How ServeIQ Addresses the Problem

- **Everything in one app.** One system for front desk housekeeping restaurant and management. When something changes in one place everyone sees it immediately.
- **Real time availability.** The system always knows which rooms are free. No more double bookings.
- **Automatic notifications.** When a guest checks out housekeeping gets notified right away. When an order is placed in the restaurant the kitchen sees it instantly on their screen.
- **Flexible pricing.** Hotel owners can set different prices for different seasons offer discount codes and create deals like early bird or last minute offers.
- **Guest self service.** Guests can search for hotels see rooms and prices and book everything from their phone. They get a QR code and confirmation without calling anyone.
- **Reports and analytics.** Real time dashboards showing occupancy revenue booking trends. Owners can make decisions based on actual data.
- **Multi property support.** One owner can manage multiple hotels from the same account.

3. System Workflow / Block Diagram

3.1 High Level System Architecture

This diagram shows how the whole system is connected from the mobile app to the backend server to the database and payment systems.

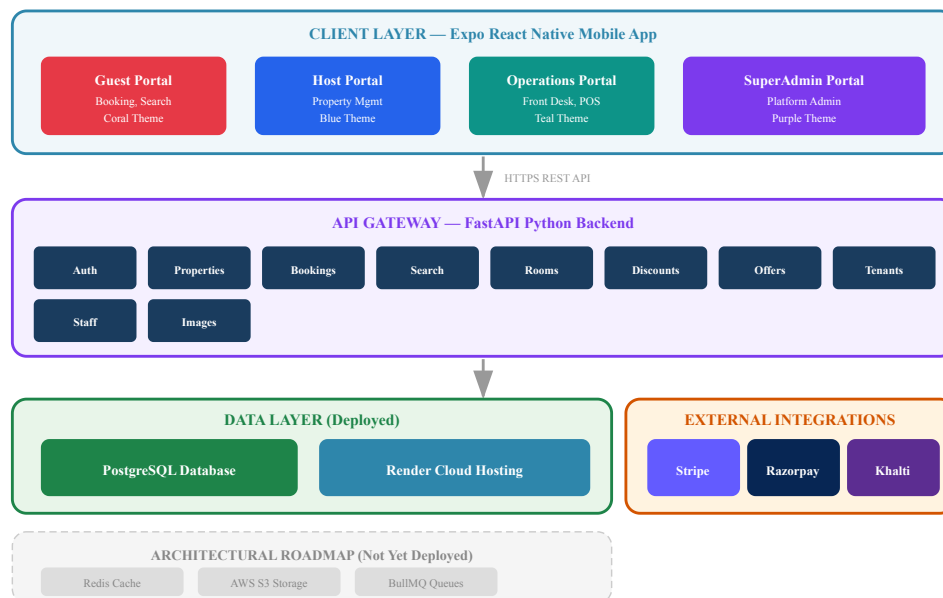


Figure 3.1 — High Level System Architecture of ServeIQ

The mobile app talks to the backend server through HTTPS REST API calls. The backend has separate routers for each feature like one for authentication one for properties one for bookings and so on. The backend stores everything in PostgreSQL which is hosted on Render. For payments

we use three gateways Stripe for international Razorpay for India and Khalti for Nepal.

Note: Some things like Redis caching AWS S3 for file storage and background job queues are planned for the future but not live yet.

3.2 Booking Flow Workflow

This is how a guest books a room from start to finish.

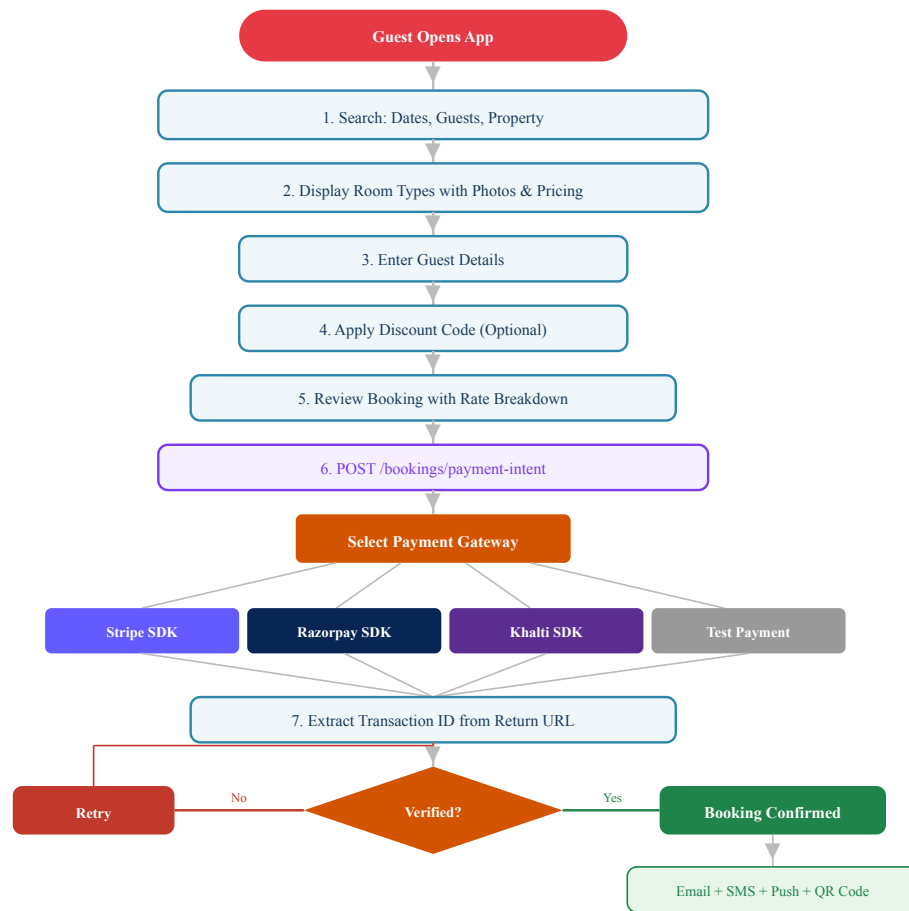


Figure 3.2 — Booking Flow Workflow

The guest searches for rooms by entering dates and number of guests. The system shows available rooms with photos and prices. The guest picks a

room enters their details and optionally applies a discount code. Then they pick a payment gateway Stripe Razorpay or Khalti and complete the payment. Once the payment is verified the booking is confirmed and the guest gets an email SMS push notification and a QR code.

3.3 Check in / Check out Flow

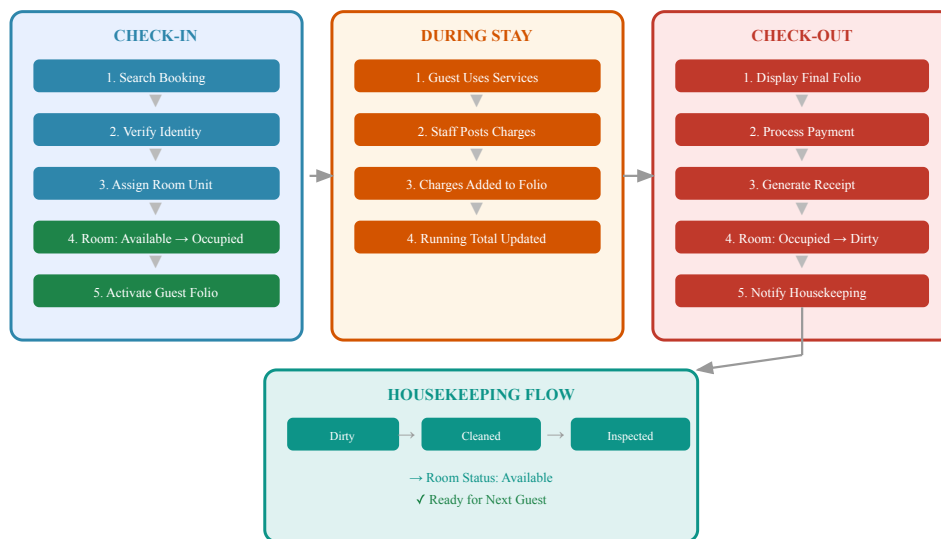


Figure 3.3 — Check in Stay and Check out Workflow with Housekeeping Flow

For check in the front desk searches for the booking verifies the guest's identity assigns a room and the room status changes from Available to Occupied. During the stay staff can add charges to the guest's folio like minibar room service or laundry. When the guest checks out the final bill is shown payment is collected and the room status changes to Dirty. Housekeeping then gets notified cleans the room and marks it as Available again.

3.4 Restaurant POS / KDS Flow

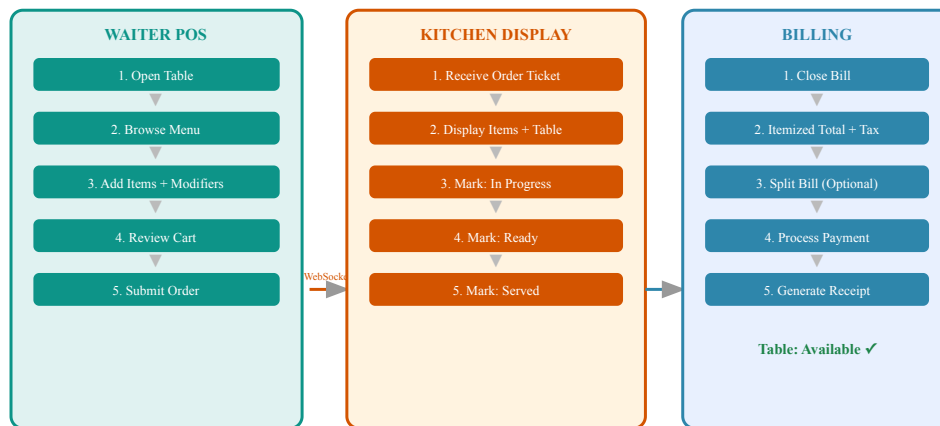


Figure 3.4 — Restaurant POS and Kitchen Display System Flow

The waiter opens a table on the POS browses the menu adds items with modifiers like size or toppings and submits the order. The order instantly appears on the kitchen display screen through WebSocket. The kitchen staff marks items as they cook them In Progress then Ready then Served. When everything is done the waiter closes the bill. The guest can pay with cash card UPI or charge it to their hotel room. After payment the table goes back to Available.

4. Description of Components / Modules

4.1 Frontend — Expo React Native Mobile Application

What it is: The mobile app that all four types of users open on their phones.

Tech used: Expo SDK 57 React Native 0.86 TypeScript Expo Router

One codebase runs on iOS Android and web. I used Expo Router which makes navigation file based. You just create files in the app folder and routes are generated automatically. State management uses Zustand for global state and React Context for portal specific stuff like auth and notifications.

Portal	Route Group	Color	Who Uses It
Guest	(tabs) /	Coral (#E63946)	People booking rooms
Host	(host) /	Blue (#2563EB)	Property owners
Operations	(operations) /	Teal (#0D9488)	Staff (front desk housekeeping restaurant kitchen)
SuperAdmin	(superadmin) /	Purple (#7C3AED)	Platform administrators

4.2 Backend — FastAPI Python Server

What it is: The server that handles all the data authentication and business logic.

Tech used: FastAPI Python hosted on Render

Router	Path	What It Does
<code>login_router</code>	<code>/auth</code>	Login register OTP verification token refresh
<code>properties_routers</code>	<code>/properties</code>	Property CRUD amenities setup wizard
<code>room_routers</code>	<code>/properties/{id}/rooms</code>	Room types and room units
<code>discount_code_router</code>	<code>/properties/{id}/discount-codes</code>	Discount code CRUD
<code>offers_routers</code>	<code>/properties/{id}/special-offers</code>	Special offers
<code>booking_router</code>	<code>/bookings</code>	Booking lifecycle payment confirmation
<code>tenants_routers</code>	<code>/tenants</code>	Tenant management (SuperAdmin)
<code>search_router</code>	<code>/search</code>	Hotel search and nearby properties
<code>image_routers</code>	<code>/properties</code>	Image upload

4.3 Database — PostgreSQL

What it is: Where all the data is stored. Properties users bookings rooms everything.

- **tenants** — The companies or people who own properties
- **users** — Staff accounts (admin manager front desk etc)
- **guests** — People who book rooms
- **properties** — Hotels resorts restaurants. Name address photos etc.
- **room_types / room_units** — Room templates and actual rooms with live status
- **bookings / booking_rooms** — Booking records and which rooms are booked
- **folios / folio_charges** — Bills and charges during a stay
- **menu_items / orders / order_items** — Restaurant menu and orders
- **staff_shifts / tasks** — Staff scheduling and task management
- **audit_logs** — Track of all important changes

4.4 Payment Gateway Integration

What it is: How guests pay for their bookings.

Gateway	Where It Is Used	How It Works
Stripe	International payments	Native SDK + WebView fallback
Razorpay	Indian market	Native SDK + WebView fallback
Khalti	Nepali market	Native SDK + WebView fallback

4.5 Operations Portal — Real Time Management

- **Front Desk:** Room grid showing which rooms are free occupied or dirty. Check in and check out processing. Folio management.
- **Housekeeping:** Task queue. Rooms go through Dirty then Cleaned then Inspected. Staff get push notifications when new tasks come in.
- **Restaurant POS:** Floor plan with tables. Waiters browse menu add items submit orders. Supports split bills and multiple payment

methods.

- **KDS (Kitchen Display):** Shows orders in real time. Kitchen staff update status as they cook. Uses color coding. Green for new orders yellow for older ones red for very late ones.

4.6 Host Portal — Property Management

- **Dashboard:** Shows stats like total properties bookings revenue.
- **Listing Wizard:** Step by step guide to set up a property.
- **Room Management:** Floor based grid to manage room types bed configuration pricing and photos.
- **Brand Customization:** Pick a brand color upload a logo.
- **Staff Management:** Invite staff assign roles manage shifts with a calendar view.
- **Discounts & Offers:** Create discount codes and special offers.

4.7 SuperAdmin Portal — Platform Administration

- **KPI Dashboard:** Total tenants monthly revenue active properties system health.
- **Tenant Management:** List all tenants suspend or activate accounts.
- **Commerce:** Manage subscription plans. Free Trial Basic Professional Enterprise.
- **Platform:** Feature flags analytics reports data exports.
- **Support:** Ticket management announcements.
- **System:** Health monitoring audit logs.

4.8 Guest Portal — Booking and Reservation

- **Home:** Shows properties in cities like Kathmandu and Pokhara.
- **Search:** Enter destination dates number of guests. Shows available rooms in real time.
- **Property Detail:** Photos room types amenities pricing reviews.

- **Booking Flow:** Multi step process. Pick room enter details pay confirm.
- **Profile:** Booking history loyalty points personal info.
- **Nearby:** Uses GPS to find hotels near the user's current location.

4.9 Push Notifications

Real time alerts sent to users' phones. When a new booking comes in the admin gets notified. When a room is checked out housekeeping gets a task. When kitchen finishes an order the waiter gets alerted. Works in foreground background and even when the app is closed.

4.10 API Client Layer

The code that handles all communication between the mobile app and the backend server. It automatically adds the right authentication token for each portal. If a request fails it retries automatically. If the backend is down it falls back to mock data so the app still works. If the user's session expires it clears the session and redirects to login.

5. Student's Role and Contribution

5.1 Overview

I have been doing my internship in Pravidhi Digital Innovations Pvt. Ltd. I am currently working as a Mobile Application Developer on the SaaS project of ServeIQ. As of now I am all alone doing the mobile app for my company with no supervision but just a QA. The most difficult thing for me was the role based login portal and catching up to the web development side of things. My favorite part of working is explaining the UI and adding random animations.

5.2 What I Did

Multi Portal Architecture

I designed the whole four portal system from scratch. Each portal has its own login its own color theme and its own set of screens. The hardest part was making sure each portal's session stays completely separate. I used portal scoped AsyncStorage keys for this.

Host Portal

I built the entire host portal. Dashboard with stats. An 8 step listing wizard. Room management with floor based grid. Brand customization with color picker and logo upload. Staff management with shift calendar. Discount and offer CRUD.

Operations Portal

I built four sub modules. Front Desk with visual room grid and check in/check out. Housekeeping with task queue and status flow. Restaurant POS with floor plan menu cart and split bill. KDS with real time order tickets and urgency color coding. I also created the shared contexts that connect these modules. For example when a waiter submits an order it shows up on the kitchen screen automatically.

SuperAdmin Portal

I built the KPI dashboard tenant management commerce module platform module with feature flags and reports support module and system module with health monitoring and audit logs.

API Integration

I connected the frontend to the live backend. I checked all 40 plus API endpoints against the backend's OpenAPI spec. I built host api with 16 plus API functions. I connected all three Operations contexts to the backend. I wired the SuperAdmin tenant screens to real endpoints. I also implemented GPS based nearby property search using the phone's location.

Payment Gateway Integration

I integrated all three payment gateways. Stripe Razorpay and Khalti. Each one has a native SDK for real devices and a WebView fallback for development. I verified end to end Khalti payment completion.

Booking Flow

I designed the multi step booking process. Room selection guest details payment gateway selection and confirmation. I also handled timezone safe date formatting discount code validation and rate breakdown calculation.

Quality Work

- Verified zero TypeScript errors after every change using `npx tsc --noEmit`.
- Tokenized about 3200 hex color references across 198 files into design tokens.
- Refactored large files into smaller modules. lib api went from one big file to 6 files. listing wizard went from 2427 lines to 712. booking flow went from 1254 to about 120.
- No new lint errors or warnings introduced by my changes. 12 pre existing issues were fixed during refactoring.

5.3 Testing and Verification

What I Tested	How	Result
TypeScript compilation	<code>npx tsc --noEmit</code>	Zero errors. All my code compiles cleanly.
Build verification	<code>CI=1 npx expo export --platform ios</code>	Passed. Metro bundler resolved all modules.
API connectivity	Cross checked with OpenAPI spec	All 40 plus endpoints verified against live backend.
Payment flow	End to end Khalti test	Payment completed successfully through native SDK.
Prebuild	<code>npx expo prebuild --no-install</code>	All native modules resolved and linked.

6. Summary and Conclusion

6.1 Project Summary

ServeIQ is a hotel and restaurant management SaaS platform built for the Nepali market. It has four portals Guest Host Operations and SuperAdmin all in one mobile app. The backend is FastAPI on Render with PostgreSQL. It supports real time operations like room status updates kitchen order tracking and push notifications. It also has three payment gateways for different markets.

6.2 How Well It Solves the Problem

Problem	How ServeIQ Fixes It
Separate systems	One app for everything. Front desk housekeeping restaurant management.
Double bookings	Real time availability. System always knows which rooms are free.
Slow staff communication	Automatic push notifications. Everyone sees changes instantly.
No smart pricing	Seasonal pricing discount codes early bird and last minute deals.
Bad guest experience	Self service portal. Search book pay from phone.
No analytics	Real time dashboards with occupancy revenue booking trends.
Cannot scale	Multi tenant SaaS. One owner can manage multiple properties.

6.3 Smart Technologies Connection

How This Relates to IoT and Smart Technologies:

Even though ServeIQ is a software platform and not a hardware IoT system it uses ideas that are the same as smart systems. The GPS based nearby search uses the phone's location to show relevant results. That is a location aware service. Push notifications are triggered automatically when something happens. A booking comes in a room gets dirty kitchen finishes an order. That is event driven like how sensors work in IoT. The room status follows a fixed state machine. Available then Occupied then Dirty then Cleaned then Inspected then Available. That is the same control logic used in smart buildings. The role based access control is like smart access management where different people get different permissions. So even though we are not using sensors the way the system is designed follows smart technology principles.

6.4 What I Learned

Technical Skills

- **React Native / Expo:** Building cross platform mobile apps that work on iOS Android and web from one codebase.
- **TypeScript:** Writing type safe code with union types generics and discriminated unions.
- **FastAPI:** Building REST APIs with Python.
- **PostgreSQL:** Database design multi tenant data isolation.
- **Payment Integration:** Stripe Razorpay and Khalti with native SDKs and WebView fallbacks.
- **State Management:** Zustand and React Context for complex multi portal state.
- **Event Driven Systems:** Real time updates for kitchen display and room status.

- **Authentication:** JWT based auth with portal scoped sessions and OTP verification.

What I Struggled With

- **Role based login:** Making sure each portal's session stays separate was the hardest part.
- **Catching up to web development:** Coming from a mobile background understanding the backend API structure was a learning curve.
- **Working alone:** As of now I am all alone doing the mobile app with no supervision but just a QA. This taught me to be more independent.

What I Enjoyed Most

My favorite part was designing the UI and adding animations. I like making things look good and feel smooth. Explaining the UI to the team and seeing it come together was the most satisfying part of this internship.

Professional Skills

- **Working independently:** Had to figure things out on my own without a senior developer guiding me.
- **Problem solving:** Debugging timezone issues payment gateway problems and React Native platform specific bugs.
- **Git and version control:** Managing code changes commits and understanding branching.
- **Documentation:** Keeping track of architecture decisions and session history.

6.5 Conclusion

This internship gave me real experience in building a full mobile application from scratch. I went from not knowing React Native to building four complete portals with real backend integration payment processing and push notifications. The most valuable thing I learned is how to work

independently and solve problems on my own. Even though it was challenging especially the role based login and catching up with the backend side I am proud of what I built. The skills I gained React Native TypeScript API integration payment gateways are things I can use in future projects and jobs.

Report Prepared By: Aswin Panta

Internship Period: July 2026 — August 2026

Organization: Pravidhi Digital Innovations Nepal Pvt Ltd

Project: ServeIQ — Hotel & Restaurant Management SaaS Platform